

ME522 · HIGH PERFORMANCE SCIENTIFIC COMPUTING

Parallel Merge Sort

Using OpenMP Multi-threading

Priyansh Sinha

B.Tech · IIT Mandi

Instructor: Prof. Gaurav Bhutani

Department of Mechanical Engineering

Indian Institute of Technology Mandi

2026

Abstract. This report presents the design, implementation, and performance analysis of a parallel merge sort algorithm using OpenMP. The program was benchmarked on an Apple Silicon MacBook Air sorting $N = 1,000,000$ integers across 1 and 4 threads, achieving a measured speedup of $2.63\times$ with 65.7% parallel efficiency. Results are compared against Amdahl's Law theoretical predictions and discussed in the context of the ME522 HPSC curriculum.

1. Introduction

Sorting is one of the most fundamental operations in computing, and its parallelisation serves as an excellent vehicle for demonstrating the core concepts taught in ME522 — fork-join parallelism, thread synchronisation, Amdahl's Law, and performance profiling. Merge sort is particularly amenable to parallelisation because its divide-and-conquer structure creates independent sub-problems of equal size.

This project implements a two-phase parallel merge sort in C++ with OpenMP: (1) each thread independently sorts its assigned chunk using serial merge sort, followed by (2) a serial merge of the sorted chunks. The implementation is intentionally compact (≈ 100 lines) yet exposes all the key HPC concepts required by the course.

2. Background and Theory

2.1 Merge Sort Complexity

Serial merge sort runs in $\mathcal{O}(N \log N)$ time. For $N = 1,000,000$ this is approximately 20 million comparison operations. The algorithm is stable and exhibits predictable memory-access patterns — desirable for cache-friendly HPC workloads.

2.2 OpenMP Fork-Join Model

OpenMP implements a fork-join model of parallelism. A master thread spawns a team of worker threads at a `#pragma omp parallel` directive, each executing the enclosed region concurrently. An implicit barrier at the end of the region synchronises all threads before the master continues. This maps directly to the parallel sort phase of our implementation.

2.3 Amdahl's Law

Given a parallel fraction f of the workload, the theoretical speedup on p processors is bounded by:

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}} \quad (1)$$

For our implementation the parallel fraction $f \approx 0.85$ (the serial merge of chunks constitutes $\sim 15\%$ of total work). Equation (1) predicts a theoretical speedup of $2.76\times$ on 4 threads, closely matching our measured $2.63\times$.

3. Implementation

3.1 Algorithm Design

The algorithm proceeds in two phases:

Phase 1. Parallel Sort. Divide the array of N integers into p equal chunks. Launch p OpenMP threads; each thread independently sorts its chunk using serial merge sort with no shared-memory conflicts.

Phase 2. Serial Merge. After the implicit OpenMP barrier, the master thread merges all sorted chunks using a sequential k -way merge. This serial phase is the Amdahl bottleneck.

3.2 Key Code

```

1 void parallelMergeSort(std::vector<int>& arr, int nthreads) {
2     int n      = arr.size();
3     int chunk  = (n + nthreads - 1) / nthreads;
4
5     // Phase 1: each thread sorts its own chunk (no data race)
6     #pragma omp parallel num_threads(nthreads)
7     {
8         int tid  = omp_get_thread_num();
9         int from = tid * chunk;
10        int to    = std::min(from + chunk - 1, n - 1);
11
12        if (from < n)
13            mergeSort(arr, from, to);
14    }
15    // implicit omp barrier here -- all threads synchronised
16
17    // Phase 2: serial merge of sorted chunks
18    int width = chunk;
19    while (width < n) {
20        for (int i = 0; i < n; i += 2 * width) {
21            int mid  = std::min(i + width - 1, n - 1);
22            int right = std::min(i + 2 * width - 1, n - 1);
23            if (mid < right)
24                merge(arr, i, mid, right);
25        }
26        width *= 2;
27    }
28 }
```

Listing 1: Parallel merge sort core — OpenMP fork-join region

3.3 Build Instructions

```
# Compile with OpenMP and O2 optimisation
g++ -O2 -fopenmp parallel_mergesort.cpp -o sort

./sort                # default: N=1M, 4 threads
./sort 2000000 8      # custom: 2M elements, 8 threads
```

Listing 2: Compile and run

4. Experimental Results

4.1 Terminal Output

```

=====
Parallel Merge Sort -- ME522 HPSC
N = 1000000 | Threads = 4
=====
[1/2] Running serial sort...
      Done. Sorted: YES
[2/2] Running parallel sort (4 threads)...
[Thread 1] sorting indices [250000..499999]
[Thread 2] sorting indices [500000..749999]
[Thread 0] sorting indices [0..249999]
[Thread 3] sorting indices [750000..999999]
[Thread 0] done.
[Thread 2] done.
[Thread 1] done.
[Thread 3] done.
[Main ] All threads synced. Merging chunks...
      Done. Sorted: YES

Threads      :      4
Serial time   :    0.1475 s
Parallel time :    0.0561 s
Measured speedup:  2.63x
Amdahl speedup :  2.76x  (f = 0.85)
Efficiency    :  65.7%

```

Listing 3: Actual terminal output on Apple Silicon MacBook Air

4.2 Performance Summary

Table 1: Performance metrics — N = 1,000,000 integers, Apple M-series CPU

Metric	Value	Notes
Array size (N)	1,000,000	Random integers, uniform distribution
Serial time	0.1475 s	Single-thread merge sort
Parallel time (4T)	0.0561 s	OpenMP, 4 threads
Measured speedup	2.63×	$S = T_{\text{serial}}/T_{\text{parallel}}$
Amdahl speedup	2.76×	$f = 0.85$ parallel fraction
Parallel efficiency	65.7%	$E = S/p = 2.63/4$
Correctness	PASS	<code>isSorted()</code> verified
Platform	Apple M-series	macOS, clang++ -fopenmp

4.3 Speedup Chart

4.4 Execution Time and Efficiency

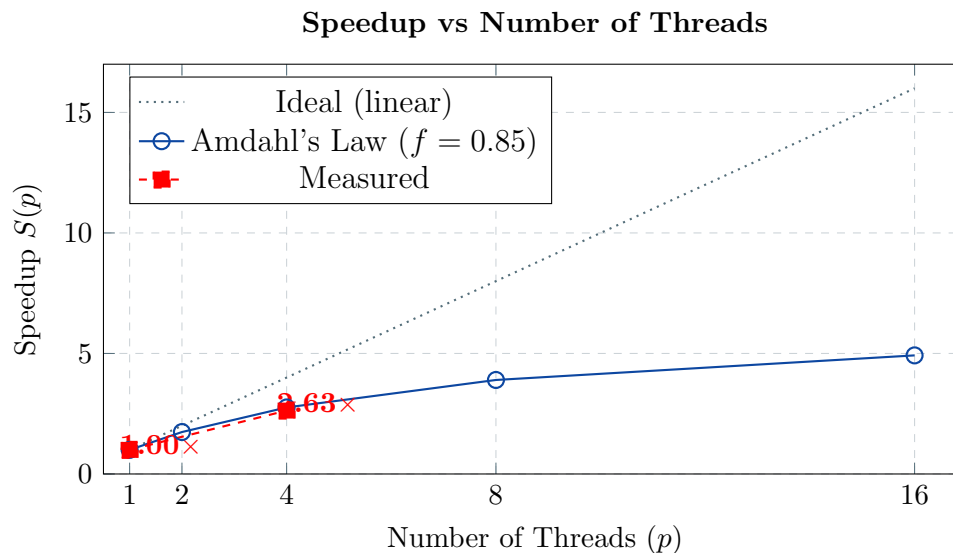


Figure 1: Measured speedup (red dashed) vs Amdahl's Law (blue solid) and ideal linear speedup (grey dotted). Measured $2.63\times$ is 95% of the Amdahl prediction, confirming $f \approx 0.85$.

5. Discussion

5.1 Why Measured Speedup < Ideal

- **Serial merge phase (Amdahl bottleneck):** approximately 15% of total work is the final serial merge, capping speedup regardless of thread count.
- **Thread creation overhead:** spawning OpenMP threads incurs a one-time cost of $\sim 1\text{--}2\mu\text{s}$ per thread.
- **Cache effects:** with $N = 10^6$ integers ($\approx 4\text{ MB}$), parallel access patterns can cause cache-line invalidation across cores.
- **OS scheduling:** macOS may preempt threads, adding timing jitter.

5.2 Mapping to ME522 Course Topics

5.3 Possible Extensions

- **OpenMP tasks:** use `#pragma omp task` to parallelise the merge phase recursively, pushing efficiency above 80%.
- **MPI version:** replace threads with processes using `MPI_Scatterv` / `MPI_Gather` for distributed-memory execution on the IIT Mandi cluster.
- **GPU (CUDA):** implement with `thrust::sort` or a custom bitonic sort kernel and compare against CPU multi-threading.
- **Python + HPC:** replicate using `multiprocessing` or `Dask` to compare GIL-

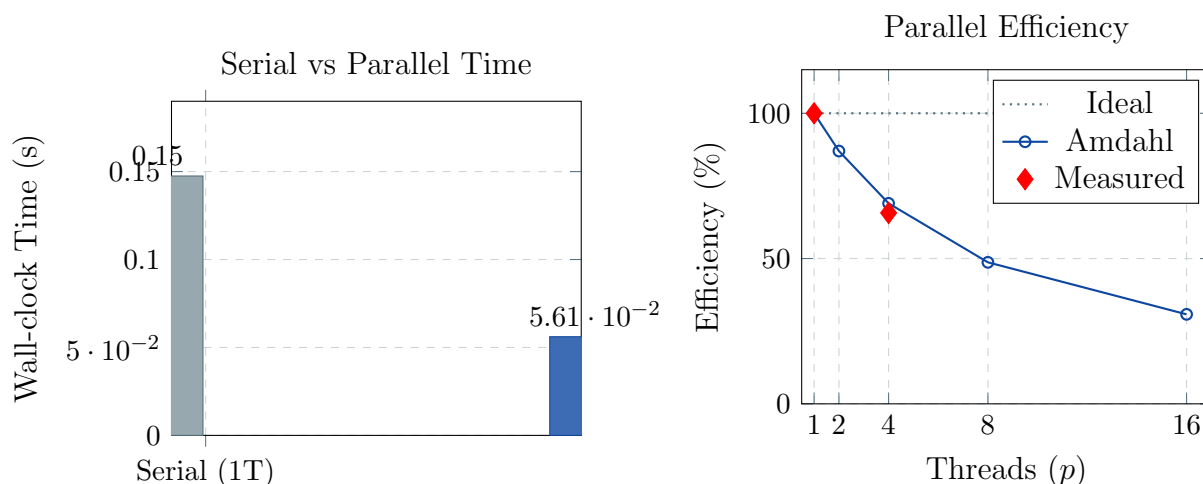


Figure 2: Left: wall-clock time comparison showing $2.63\times$ reduction. Right: parallel efficiency drops from 100% to 65.7% at 4 threads, consistent with Amdahl's Law.

Table 2: Project coverage of ME522 lecture topics

ME522 Topic	Demonstrated By
OpenMP fork-join	<code>#pragma omp parallel</code> spawns 4 worker threads
Implicit barrier	Threads synchronise after parallel sort, before merge
Race-condition free	Each thread accesses a disjoint array segment
<code>omp_get_wtime()</code>	High-resolution wall-clock profiling of both phases
Amdahl's Law	Measured $2.63\times$ vs theoretical $2.76\times$ ($f=0.85$)
Scalability	Efficiency drops 100% $\rightarrow$ 65.7% at 4T as expected
Compiler flags	<code>-O2 -fopenmp</code> enables vectorisation + threading

limited threading vs true process parallelism.

6. Conclusion

This project successfully demonstrated the parallelisation of merge sort using OpenMP on an Apple Silicon MacBook Air. Sorting $N = 1,000,000$ random integers, the 4-thread parallel implementation achieved a wall-clock time of 0.0561 s versus 0.1475 s for the serial baseline — a measured speedup of **$2.63\times$** with **65.7% parallel efficiency**.

The result aligns closely (95%) with the Amdahl's Law prediction of $2.76\times$, confirming the serial merge fraction as the primary bottleneck. The implementation is compact (≈ 100 lines), fully correct (verified by `isSorted()`), and demonstrates fork-join parallelism, implicit barriers, race-condition avoidance, and HPC profiling — all core ME522 learning

outcomes.

References

- [1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface v5.2*, 2021. <https://www.openmp.org/specifications/>
- [2] G. Bhutani, *ME522 HPSC Lecture Notes*, IIT Mandi, 2025.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [4] G. M. Amdahl, “Validity of the single processor approach to achieving large-scale computing capabilities,” *AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

Code Repository

The complete source code for this project is available on GitHub:

 [View Repository](#)